LP3 Assignment B3

Roll no: 41168

Problem Statement:
Given a bank customer, build a neural network-based classifier that can determine whether they will leave or not in the next 6 months.
Dataset Description: The case study is from an open-source dataset from Kaggle. The dataset contains 10,000 sample points with 14 distinct features such as CustomerId, CreditScore, Geography, Gender, Age, Tenure, Balance, etc.
Dataset Link: Churn Modelling Dataset (Kaggle)
https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling
Perform following steps:
1. Read the dataset.
2. Distinguish the feature and target set and divide the data set into training and test sets.
3. Normalize the train and test data.
4. Initialize and build the model. Identify the points of improvement and implement the same.
5. Print the accuracy score and confusion matrix (5 points).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

import tensorflow as tf
tf.keras.utils.set_random_seed(1)

# 1. Read the dataset (Kaggle: Churn_Modelling.csv)
df = pd.read_csv('Churn_Modelling.csv')
df.head()
```

[1] ✓ 7.6s Python

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	H
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	

```python
# 2. Features / target, drop identifiers, encode categoricals, split
X = df.drop(columns=['RowNumber', 'CustomerId', 'Surname', 'Exited'])
X = pd.get_dummies(X, columns=['Geography', 'Gender'], drop_first=True).astype('float32')
y = df['Exited']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

[2] ✓ 0.0s Python

```python
# 3. Normalize (fit on train only)
print('BEFORE scaling (train):')
print(X_train.agg(['mean', 'std', 'min', 'max']).T.round(2))

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

print('\nAFTER scaling (train): max |mean| =', abs(X_train.mean(axis=0)).max().round(6),
      '| std range =', X_train.std(axis=0).min().round(3), 'to', X_train.std(axis=0).max().round(3)
print('AFTER scaling (test) : mean =', X_test.mean().round(3), '| std =', X_test.std().round(3),
      '(uses train statistics, so not exactly 0/1)')
print('Scaler fitted on', int(sc.n_samples_seen_), 'train samples only')
```

[3] ✓ 0.0s Python

```
BEFORE scaling (train):
                        mean           std     min            max
CreditScore          650.750000     96.599998  350.00     850.000000
Age                   38.950001     10.530000   18.00      92.000000
Tenure                 5.020000      2.900000    0.00      10.000000
Balance            76381.210938  62302.058594    0.00  238387.562500
NumOfProducts          1.530000      0.580000    1.00       4.000000
HasCrCard              0.710000      0.450000    0.00       1.000000
IsActiveMember         0.510000      0.500000    0.00       1.000000
EstimatedSalary    99730.812500  57335.121094   11.58  199992.484375
Geography_Germany      0.250000      0.430000    0.00       1.000000
Geography_Spain        0.250000      0.430000    0.00       1.000000
Gender_Male            0.550000      0.500000    0.00       1.000000

AFTER scaling (train): max |mean| = 0.0 | std range = 1.0 to 1.0
AFTER scaling (test) : mean = -0.006 | std = 1.0 (uses train statistics, so not exactly 0/1)
Scaler fitted on 8000 train samples only
```

```python
# 4a. Baseline model
base = Sequential([
    Dense(6, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(6, activation='relu'),
    Dense(1, activation='sigmoid')])
base.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
base.fit(X_train, y_train, epochs=50, batch_size=32, verbose=0)

pred = (base.predict(X_test) > 0.5).astype(int)
print('Baseline accuracy:', accuracy_score(y_test, pred))
print(confusion_matrix(y_test, pred))
```

[4] ✓ 24.1s Python

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Baseline accuracy: 0.8655
[[1544   49]
 [ 220  187]]
```

```python
# 4b. Improved model: wider layers, dropout, early stopping on validation loss
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(1, activation='sigmoid')])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=200, batch_size=32, validation_split=0.2,
          callbacks=[EarlyStopping(patience=25, restore_best_weights=True)], verbose=0)

pred = (model.predict(X_test) > 0.5).astype(int)
print('Improved accuracy:', accuracy_score(y_test, pred))
print(confusion_matrix(y_test, pred))
```

[5] ✓ 33.5s Python

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Improved accuracy: 0.865
[[1531   62]
 [ 208  199]]
```